

OpenAI Agents API 小白指南

把开源 Codex harness 变成云 API —— 2026-09-10 发布, Agent Harness 从"自己拼"变"直接租"



Agent Harness 即服务



Session 持久资源



Subagent 编排



自动上下文压缩



三态沙箱



开源 131 crates

3 层

运行时选择: Agents API
/ SDK / Responses

4 概念

Agent · Environment ·
Session · Events

5 能力

压缩恢复 · 工具MCP ·
Subagent · 沙箱 · 接自有
机器

131 crates

开源 codex 仓库 5094 个
Rust 文件 200 万行

? 工科毕业生的 12 个疑问 (疑问即目录)

1. Agents API 到底是什么? "又一个 Agent SDK"?
2. 我会 LangGraph/CrewAI 了, 为什么要"租 harness"?
3. Agent / Environment / Session / Events 怎么咬合?
4. 三层运行时 (API / SDK / Responses) 怎么选?
5. "长任务自动压缩恢复"是怎么做到的?
6. Subagent 并行编排长什么样?
7. 沙箱三态是什么? self-hosted 怎么连?
8. 程序化工具调用被官方产品化了?
9. harness 开源意味着什么? 131 个 crate 里都是啥?
10. 怎么计费? 比直接调模型贵多少?
11. 安全边界: key 放哪? 数据在哪?
12. 我能用它干什么? 什么时候不该用?

Q1 Agents API 到底是什么？"又一个 Agent SDK"？

一句话：**OpenAI 把自家 Codex 背后的 Agent harness 原样搬进云端，变成一个 REST API**。你的应用只管发任务、收事件；模型循环、上下文管理、断点恢复、沙箱运维，全由 OpenAI 代管。

它和 SDK 的本质区别在"循环跑在哪"：SDK 是把一个库装进你的进程，**agent 循环在你的代码里跑**；Agents API 是**循环根本不在你机器上**——OpenAI 云端跑着一个托管版 Codex 实例（官方叫 it "managed Codex harness"），你通过 HTTP 跟它对话。API 端点是 `POST /v1/agents/sessions`，需要 `OpenAI-Beta: agents=v1` 头，目前处于 **public beta**。



官方自己的分界线（developers.openai.com 文档原话）：Agents API = "OpenAI runs a managed Codex harness"; Agents SDK = "The SDK runs inside your application"。一个是**租整条生产线**，一个是**买机器回家自己开**。

Q2 我会 LangGraph / CrewAI 了，为什么还需要"租 harness"?

先讲一个之前的世界：2025 年起人人都发现 **模型本身不难接，难的是模型外面那一圈**——上下文窗口满了怎么办？任务跑 3 小时进程挂了怎么办？工具执行环境怎么隔离？OpenAI 官方博客《Harness engineering》的原话：agent 反复失败，大多不是模型不行，而是**环境欠规格**（这正好印证知识库 [[AgentHarness智能体挽具]] 的七职责：改环境胜过改形容词）。

于是每家公司都在重复造这一圈。Agents API 的赌注是：**这圈东西 OpenAI 已经在 Codex 里打磨好了，别再造了，直接调用**。你写 `client.beta.agents.sessions.create(...)`，OpenAI 替你把压缩、恢复、subagent 调度、沙箱生命周期全做了。

LangGraph / CrewAI (已学)		Agents API (本篇)
编排逻辑	你写：状态图 / 角色分工	harness 自主决定（可多给指令引导）
状态持久化	你配 checkpointer ([[Checkpoint存档与持久执行]])	内建：session 自动保存 turns/items
执行环境	你的进程 + 你选的沙箱方案	三态沙箱开箱即用
上下文管理	你写压缩策略 ([[上下文预算与战略压缩]])	自动压缩内建
换模型/厂商	自由（可接任意 LLM）	锁 OpenAI 生态 (gpt-6-astra 等)
适合	要精细控制流程的产品	长时自主任务、不想运维 agent 基建

记住这组对照：LangGraph 给你"流程图画笔"，Agents API 给你"一个已经会干活的员工"。前者赢在可控，后者赢在省事——代价是控制权。

Q3 Agent / Environment / Session / Events 四概念怎么咬合?

官方文档把 API 建在四个对象上，类比一个外包员工的工作档案：

概念	是什么	类比	在请求体里
Agent	模型+指令+工具+MCP 的配置包	岗位说明书	<code>agent: {...}</code> 或复用 <code>agent_id</code>
Environment	可选的沙箱/电脑：跑命令、放文件、装技能	工位	<code>environment: {type: ...}</code>
Session	持久化的 agent 实例：干活的主体，跨轮次存活	员工本人在册	创建后返回 <code>sess_...</code> id
Events & Items	你发进去的输入、它干出来的每一步记录	工作日志	stream 事件 / <code>sessions.items</code>

Session 的一生（状态在 OpenAI 那边，关掉你的电脑它还活着）



易混警告：Session ≠ SDK 的 session ≠ Responses conversation ≠ sandbox，是四种不同资源。SDK 的 session 是你进程里的对象；Agents API 的 session 是**存在 OpenAI 服务端的持久资源**，可以 delete、retrieve、跨天继续。这是“会话即资源”（session as a resource）的思路——对照知识库 [[Checkpoint存档与持久执行]] 里 thread_id 一人一档的思路，OpenAI 把它做成了第一等 API 资源。

Q4 三层运行时（Agents API / Agents SDK / Responses API）怎么选?

OpenAI 现在有三条做 agent 的路，官方对比表的“人话版”：

	Agents API	Agents SDK	Responses API
用一句话选它	长任务，OpenAI 替我管进度	agent 循环要在我的应用里	只要模型响应，循环自己写
循环跑在哪	OpenAI 云（托管 Codex）	你的进程	你的进程
接入成本	低	中	高
任务间状态	session 自动存	你自己的存储	手动拼历史
执行环境	官方沙箱/自有沙箱/无	你的运行时	你自己的环境

✅ **决策口诀：**要“全自动代驾”→ Agents API；要“方向盘在我手里”→ Agents SDK（即知识库里 [[智能体框架]] 的角色，OpenAI 版）；只要“发动机”→ Responses API。注意别和 `openai-agents` 这个 pip 包搞混——那是 2025-03 的 Agents SDK，和新 Agents API 是两代东西（本机实测：装了 SDK 3.24.0 后 openai-agents 0.20.0 报依赖冲突，两代并存是个坑）。

Q5 "长任务自动压缩恢复"是怎么做到的？

长任务的死穴是上下文窗口：干几小时，历史记录爆窗。Agents API 的答案是**托管压缩 (automatic context compaction)**——harness 把"总结之前的工作"当成常规操作，窗口快满就自动摘要旧进展、保留关键状态，任务不中断。推文说的"长任务自动上下文压缩恢复"就是它。

我们在开源 codex 仓库里找到了它的源码实锤（见 ex4）：

```
// codex-rs/core/src/compact.rs — 压缩是一等公民，不是补丁
use codex_analytics::CompactionReason; // 为什么压：自动触发 or 手动
use crate::context::CompactionSummary; // 压出来的摘要对象
// 配套：compact_token_budget.rs / compact_remote_v2.rs /
//       compact_model_fallback.rs (压缩模型降级)
```

恢复则是另一半：session 的 turns/items 存在服务端，流断了可以 retrieve 回来续传（官方文档专门有一节 "recover a disconnected stream"），SDK 里 `with_result_collection()` 最后一帧给 final result + session_id。

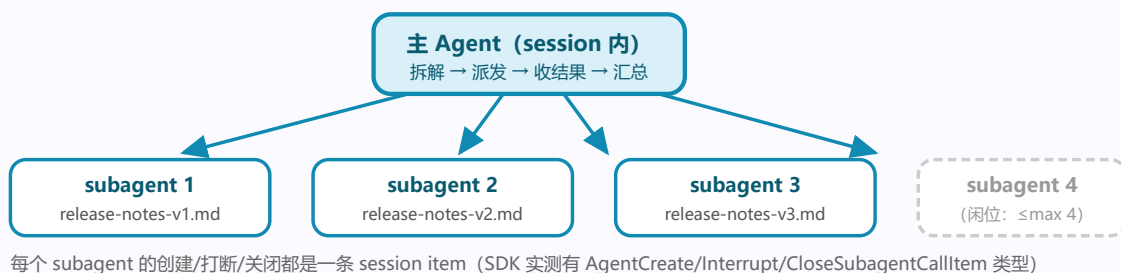
对照已学：[[上下文预算与战略压缩]] 讲"50% 早压缩、逻辑断点才压、实现中千万别压"——这些是**你自己写 harness 时的经验法则**；Agents API 把同类策略做成了**托管默认值**，你不用再管。恢复侧对应 [[Checkpoint存档与持久执行]]（崩了从存档续跑不重花钱），区别：LangGraph 存在你配的 checkpointer 里，这里存 OpenAI 服务端。

Q6 Subagent 并行编排长什么样？

创建 session 时打开一个开关，主 agent 就获得了"招临时工"的能力：

```
"multi_agent": { "enabled": true, "max_concurrent_subagents": 4 }
```

之后 harness 在干活时会**自己决定**把独立子任务拆给并行 subagent（官方示例：对比多份 release notes，每份一个 subagent 并行读，最后汇总）。分工方式不是你画 DAG，而是 harness 自主拆解——这是和 [[工单任务图与前沿调度]]（你显式建模任务依赖）最大的差别。



对照已学：[[多智能体协作]] (crewAI 角色分工) 是"你指定谁出场"；这里 harness 自己招人裁员，你只设并发上限。坑也对应 [[多Agent协作乱序竞态]]：并行的完成顺序≠依赖顺序，汇总逻辑仍要防"摘要先于数据定稿"。源码坐标：`core/src/agent/control/execution.rs`（实测命中 `SessionSource::SubAgent(_)` 分支）。

Q7 沙箱三态是什么？self-hosted 怎么连？

Environment 有三种形态，对应三种架构（官方 architecture 文档）：

type	工位在哪	能干什么	适合
<code>none</code>	没有工位	回答问题、调远程 MCP / function 工具； 没有 Bash、apply-patch、工作区文件	问答、调研型 agent
<code>openai_hosted</code>	OpenAI 云沙箱	跑 Python/Node/CLI、改文件、产 artifacts；可配网络权限和依赖包	写代码、跑脚本的通用型
<code>self_hosted</code>	你的机器 （笔记本/Docker/Lambda）	你的代码起 executor 连上 session，harness 远程下发命令由 executor 执行	私有网络、内网数据库、专有软件

self-hosted 的精髓：**命令不是你转发给 harness 的，而是 harness 下发给你的 executor**。你的代码只管 executor 的生死（开机、断线重连、关机），不需要逐条命令过手。key 永远不进沙箱（官方反复强调）。

✅ **本机实测（ex5，零费用）**：装 `npm i -g @openai/codex` (0.160.0)，`codex sandbox python -c "print('hello')"` 在 Windows restricted token 沙箱内成功输出；同环境尝试写文件 → `PermissionError: [Errno 13] Permission denied`，文件确实没落盘。**这套沙箱和云上是一个开源实现**——开源仓库里 sandbox 家族有 5 个 crate: linux-sandbox / windows-sandbox-rs / windows-sandbox-service / mxc-sandbox / sandboxing。

Q8 程序化工具调用（Programmatic Tool Calling）被官方产品化了？

是的——正是知识库 [[程序化工具调用]]（CodeAct，cmu-agents 批次学的“模型写代码进沙箱一次跑完整策略”）的**官方 API 版**。创建 session 时声明：

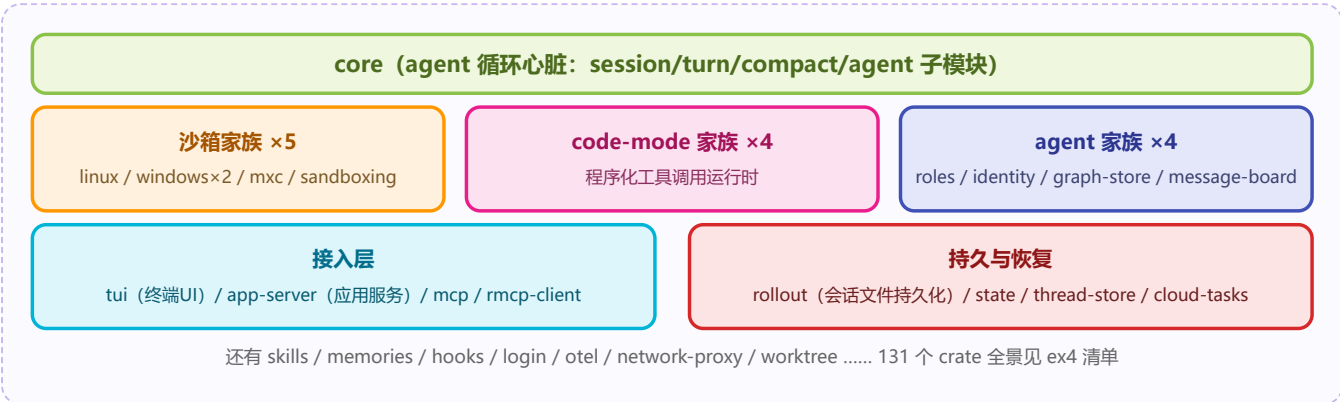
```
"tools": [ { "type": "programmatic_tool_calling" } ]
```

打开后 agent 不再“每轮一个工具调用”，而是**写一段代码把多个工具调用编排成一次执行**（枚举→模拟→评估→择优→行循环搞定）。开源仓库里对应一整个 crate 家族：`code-mode / code-mode-host / code-mode-protocol / code-mode-runtime` + `tools/src/code_mode.rs`。

定位：CMU 作业里它是“自己搭的教学版”，现在 OpenAI 直接把它做成了 `{"type": "programmatic_tool_calling"}` 一个开关。**从论文概念到产品开关只用了不到一年**——这是本篇最值得记住的行业信号：知识库那些“模式”正在快速变成云厂商的 API 参数。

Q9 harness 开源意味着什么？131 个 crate 里都是啥？

发布的核心事实：云上跑的 harness 和你电脑上能装的 Codex CLI 是同一套开源代码（openai/codex, Apache-2.0, 127k+ stars）。官方公告原话 "backed by the Codex harness" + "view the public codebase"。实测仓库规模：**131 个 Rust crate、5094 个 .rs 文件、约 200 万行代码。**



开源意味着三件事：① **可审计**——云上 agent 到底执行了什么逻辑，源码可查（本篇 ex4 就这么干的）；② **可自建**——同一套 harness 可以本地跑（CLI），云 API 只是托管形态；③ **可学习**——这就是市面上最完整的"生产级 harness 教科书"，[[AgentHarness智能体挽具]] 七职责每一条都能在里面找到对应 crate。

Q10 怎么计费？比直接调模型贵多少？

官方 pricing 一句话：**模型按模型价、工具按工具价、沙箱按容器价——没有"Agents API 附加费"**。具体三笔账：

账目	怎么算
模型推理	所选模型的标准 API 价（gpt-6-astra 等）
OpenAI 内建工具	web_search 等按 built-in tools 标准价
托管沙箱	openai_hosted 按容器（container）标准计费—— 沙箱开着就烧钱

风险点和 [[交易成本与鞭打效应]] 同理：**账单放大器藏在"agent 循环圈数 × 沙箱时长 × subagent 并发数"**。一次 session 主 agent 拆 4 个 subagent，token 消耗就是 5 条循环相加。session 记账有专门的 observability 端点（sessions.traces / usage），上线前先看这份账。

Q11 安全边界：API key 放哪？数据在哪？

四条红线，全部来自官方文档原文：

- ① **key 权限最小化**：会话操作要 `api.agents.read` + `api.agents.write`，模型推理要 `api.responses.write`——用应用级 key，别拿全家桶 key。
- ② **key 永不进沙箱**：官方 quickstart 原话 "Keep this key outside the agent's sandbox" (agent 能执行任意代码，key 进去等于送给它)。
- ③ **数据驻留仅美国**：目前只支持 US 数据驻留。
- ④ **不支持 ZDR**：Zero Data Retention 不适用，选 self_hosted 沙箱也改变不了这一点——harness 和会话状态始终在 OpenAI 侧。合规敏感场景要过一遍这条。

实测彩蛋 (ex2)：无 key 请求返回 `"A valid actor biscuit is required"`——"actor biscuit" 是 OpenAI 内部鉴权层的术语，从错误信息能窥见服务端身份模型 (actor = 发起动作的主体)。对照 [[控制面与数据面]]：认证闸门在控制面，一进 API 就验。

Q12 我能用它干什么？什么时候不该用？

能力清单（学完你手上多了什么）：

- ① **会话即资源**的心智模型：把"一个长期干活的 agent"当数据库记录管理 (create/retrieve/update/delete)；
- ② **三态沙箱架构图**：none / openai_hosted / self_hosted 的选型判断；
- ③ **一套可读的生产级 harness 源码** (131 crates)，七职责逐条有实码；
- ④ **本地 codex sandbox**：零费用给任何命令套受限令牌沙箱；
- ⑤ **subagent 并发上限思维**：multi_agent.max_concurrent_subagents 一个参数管并行度。

✓ 适合用	✗ 不适合
长时自主任务（调研、批量改代码、事故排查）	要接非 OpenAI 模型（锁生态）
需要断点续跑、服务端状态的 agent 产品	要精细控制每步流程（选 Agents SDK / LangGraph）
个人项目快速获得"会干活的 agent"不想运维基建	数据不能出内网/不能存美国（no ZDR）
内网自动化（self_hosted executor 连私有网络）	高频短调用（沙箱容器费可能比模型费还贵）

给你的行动建议（结合你已会的）：先用 **local CLI + sandbox**（免费部分）体会 harness；要写 agent 产品时拿 quickstart 的 session 流程对接自己的 webhook；架构选型时把本篇 Q4 三层表和 [[Agent设计模式目录]] 的 Level 0-3 对号入座。

 **动手实验：5 个零 key 零费用实验（全部真实运行）**

实验 ex1 · 官方 SDK 内省取证 —— 证明 Agents API 真实存在

步骤1 升级 SDK 并内省命名空间

`pip install --upgrade openai` → `python ex1_sdk_introspect.py`。目的：验证 `beta.agents` 命名空间真实存在于官方 SDK。

✓ **预期/实测：**2.54.0 报 `'Beta' object has no attribute 'agents'`；升级到 3.24.0 后 `beta.agents` OK，sessions 子资源 `turns/events/items/subagents/artifacts/traces/stream` 全部存在；`openai.types.beta` 中 agent 相关类型 **161 个**。

步骤2 看 sessions.create 签名（打印自动给出）

✓ **实测关键参数：**`agent` 或 `agent_id`（复用型 Agent 资源）、`environment`、`input`、`stream`、`vault_ids`、`output_type`（类型化输出！）。subagent 生命周期三件套 `AgentCreate/Interrupt/CloseSubagentCallItem` 都是 item 类型。

实验 ex2 · 无 key 真请求 —— 端点存在性探测

操作 `python ex2_nokey_probe.py`（httpx 向 `/v1/agents/sessions` 发三种请求）。目的：不花钱验证端点真实、鉴权在控制面。

✓ **实测：**无 key → HTTP 401 `"A valid actor biscuit is required"`；假 key → HTTP 401 `"Incorrect API key provided"`；对照组不存在端点 → HTTP 404。**401 vs 404 证明端点活着且认证前置。**

实验 ex3 · Session 请求体构造器 + 契约自校验

操作 `python ex3_session_builder.py`。目的：亲手把四概念翻译成请求体 JSON，并实现环境三态/工具枚举/并发参数校验。

✓ **实测：**三架构场景（问答 none / 编码 `openai_hosted` / 私网 `self_hosted+MCP+multi_agent`）契约检查全 PASS；负向用例（none 环境带 workspace）被正确拦截，报错 `none 环境没有工作区（无 Bash/apply-patch/executor MCP）`。产出 `session_bodies.json` 三份合法请求体。

实验 ex4 · 开源 harness 源码取证 —— 五能力坐标地图

操作 `git clone --depth 1 https://github.com/openai/codex.git repo` → `python ex4_harness_forensics.py`。目的：把推文吹的五大能力逐个钉到源码行号。

✓ **实测全 FOUND：**仓库 131 crates / 5094 个 .rs / **2,002,055 行**。①压缩→`core/src/compact.rs:29`（`CompactionReason`）；②工具MCP→`rmcp-client/src/lib.rs:29`；③subagent→`core/src/agent/control/execution.rs:85`（`SessionSource::SubAgent`）；④沙箱→`sandboxing/src/lib.rs:21`（`mxs sandbox` 参数）；⑤云端恢复→`rollout/src/lib.rs:1`（`session` 文件持久化）。

实验 ex5 · 本地 Codex CLI 沙箱真跑（本篇成就感实验）

步骤1 安装： `npm install -g @openai/codex` → `codex --version`

✓ 实测输出 `codex-cli 0.160.0`（npm 包与开源仓库同源）。

步骤2 沙箱内跑命令： `codex sandbox python -c "print('hello from inside codex sandbox'); import sys; print(sys.platform)"`

✓ 实测输出 `hello from inside codex sandbox / win32` ——**Windows restricted token 沙箱真实生效，零模型零费用。**

步骤3 验证隔离性： `codex sandbox python -c "open('x.txt','w').write('x')"`

✓ 实测 `PermissionError: [Errno 13] Permission denied`，`x.txt` 确实未创建——**agent 写权限被硬件级拦住，这就是 harness “最小权限”职责的实物教学。**

✓ **成就达成：** 你没花一分钱 API 费，却亲手跑进了 OpenAI 生产 harness 的沙箱内部，还亲眼看见它的权限墙把你弹回来。

📌 一页记忆卡

是什么： Agents API = 托管版 Codex harness (2026-09-10 public beta) ——你的应用发任务收事件，模型循环/压缩/恢复/编排/沙箱全在 OpenAI 云端。

四概念： Agent（岗位说明书）· Environment（工位：none/openai_hosted/self_hosted 三态）· Session（在册员工，sess_id 持久资源）· Events & Items（工作日志）。

五能力： 自动上下文压缩恢复 / 原生 MCP+自定义工具 / subagent 并行（max_concurrent_subagents） / 托管沙箱 / self-hosted executor 连自有机器。

三层怎么选： 要代驾→Agents API；要方向盘→Agents SDK；只要发动机→Responses API。

两个提醒： 计费=模型价+工具价+**沙箱容器时长**（真正的账单放大器）；数据仅美国驻留、**不支持 ZDR**，key 永不进沙箱。

与本库知识的接口： [[AgentHarness智能体挽具]]（七职责全对上 131 crates）· [[程序化工具调用]]（官方产品化开关）· [[Checkpoint存档与持久执行]]/[[上下文预算与战略压缩]]（托管化了）· [[工单任务图与前沿调度]]（对照：harness 自主拆解 vs 你画 DAG）。

信息来源： 推文 (MaxForAI 2098334720677282098, 引 OpenAI 员工 stevendcoffey) → 官方公告

openai.com/index/introducing-the-agents-api/ → 官方文档 developers.openai.com/api/docs/guides/agents-api/*

（overview/quickstart/architecture, 2026-10-03 抓取）→ 开源仓库 github.com/openai/codex (Apache-2.0, 131 crates 实测取证)。实验环境：Windows 11 / Python 3.14.7 / openai 3.24.0 / codex-cli 0.160.0。全部实验零 key 零费用真实运行。